

DOCUMENTO ESTRATÉGICO · AUTENTICAÇÃO

Autenticação, Login & Segurança ligados ao Banco de Dados

Cadastro, login, recuperação de senha, 2FA, LGPD e JWT — com tabelas e colunas do PostgreSQL. 8 páginas.

ESCOPO

users · user_settings · user_consent · subscriptions · reset/2FA projetados · Stripe · admin

BANCO

PostgreSQL Railway · bcrypt · JWT HS256 · isolamento por user_id

EQUIPE

Davi Almeida
Leonardo Sena
Gustavo Biscoto

Agosto / 2026
Curitiba — PR

1

Visão Geral da Autenticação

Onde a senha fica · Como a sessão funciona · Isolamento

Este documento responde: onde a senha fica? Como ninguém vê meus dados? O que é “esqueci senha”? E se o banco vazar?

4

Tabelas auth

2

Tabelas futuras

10

Rounds bcrypt

7d

JWT

Senhas irreversíveis

bcrypt 10 rounds em users.password_hash. Login: compare — sem decrypt.

Sessão JWT

HS256 com JWT_SECRET. Payload mínimo {userId, email}. Bearer em toda rota.

Regra de ouro

Toda query: WHERE user_id = \$jwtUserId. Role app sem SUPERUSER / sem DROP.

Pilar	Protege	Onde	Como
bcrypt	Senhas	users.password_hash	10 rounds · compare
JWT	Sessão	.env JWT_SECRET	HS256 · 7 dias
user_id	Isolamento	todas as tabelas	authPreHandler
Rate limit	Brute force	Fastify	5 tent. / 15 min

2

Login passo a passo

Do formulário ao dashboard · cada query

- 1 **POST /auth/login**
JSON {email, password}; validação de formato no front.
- 2 **Zod no backend**
Rejeita payload inválido sem tocar no Postgres (HTTP 400).
- 3 **SELECT users**
WHERE email=\$1 via UNIQUE. Sem linha → 401 genérico.
- 4 **JOIN settings/subscription**
LEFT JOIN para onboarding, tema e plano ativo.
- 5 **bcrypt.compare**
Falso → 401; rate limit após 5 falhas.
- 6 **jwt.sign**
HTTP 200 {token, user, settings...}; front guarda localStorage.

Tabela	Coluna	Tipo	Uso
users	email	TEXT	Busca UNIQUE
users	password_hash	TEXT	bcrypt
users	plan	ENUM	Recursos
user_settings	onboarding_completed	BOOL	Fluxo 1º acesso
subscriptions	status	ENUM	Plano pago
user_consent	document_version	TEXT	Reaceite LGPD

3

Cadastro + conformidade LGPD

4 INSERTs · terms / privacy / lgpd

Antes do formulário: GET /auth/legal entrega textos e versões. Cadastro = 4 INSERTs em 1 transação ACID.

Prova de aceite

user_consent

guarda data, IP, UA e versão. UNIQUE(user, type, version).

Tudo ou nada

Falha no UNIQUE(email) → ROLLBACK. Sucesso → COMMIT + JWT.

- 1

BEGIN

Transação PostgreSQL.
- 2

INSERT users

UUID, name, email, hash, phone, plan=free.
- 3

INSERT user_settings

1:1 com alertas padrão e onboarding=false.
- 4

3× user_consent

terms · privacy · lgpd com versão atual.
- 5

COMMIT + JWT

Mesma resposta do login.

consent_type	Documento	Colunas de auditoria
terms	Termos de Uso	agreed_at, ip_address, user_agent, document_version
privacy	Política de Privacidade	idem
lgpd	Consentimento LGPD	idem

4

Recuperação de senha (projeto)

Modelagem pronta no PostgreSQL

Ainda não está na API, mas o desenho do banco já evita rework — mostra visão de segurança na banca.

- 1
- POST /auth/forgot**
Resposta genérica (não revela se o e-mail existe).
- 2
- INSERT password_reset_tokens**
Guarda SHA-256 do token · expires 30 min · used=false.
- 3
- E-mail com link**
/reset?token=... · verify calcula hash e valida.
- 4
- Nova senha**
UPDATE password_hash · token_version++ · used=true (invalida JWTs velhos).

Coluna	Tipo	Regra
id	UUID	PK
user_id	UUID	FK CASCADE
token_sha256	TEXT	Nunca o token puro
expires_at	TIMESTAMPTZ	+30 min
used / used_at	BOOL / TSTZ	Uso único
ip / user_agent	TEXT	Auditoria LGPD

Por que hash do token?
Vazamento da tabela não monta o link válido — defesa em profundidade.

5

Validação em 2 etapas (projeto)

App autenticador · SMS · e-mail

Etapa	O que acontece	Tabela
Ligar 2FA	Gera secret base32 + QR (TOTP RFC 6238)	two_factor_secrets
Confirmar	1º código válido → flag ligada	user_settings.two_factor_enabled
Login + desafio	Após senha OK, ainda sem JWT	two_factor_challenges
Código	≤3 tentativas / 2 min → JWT final	attempts, expires_at

two_factor_secrets

Coluna	Tipo
user_id	UUID PK→users
secret_base32	TEXT cifrado
method	app/sms/email
backup_codes_hash	JSONB

two_factor_challenges

Coluna	Tipo
id	UUID
user_id	FK CASCADE
code_hash	bcrypt
expires_at / attempts	+2min / ≤3

Na banca

Mostre que o schema já prevê MFA sem redesenhar users — só novas tabelas 1:1 / 1:N.

6

Planos e billing no banco

ENUM users.plan · tabela subscriptions

Quem muda o plano é o **webhook Stripe** (HMAC), não um botão do painel.

Plano	Valor	Enum	Libera
Free	R\$ 0	free	Limite mensal · WhatsApp opcional
Pro	R\$ 19,90	pro	Ilimitado · PDF · chat básico
Premium	R\$ 49,90	premium	WhatsApp oficial · admin · logs IA

- 1

Checkout
Stripe coleta cartão → customer_id em users.
- 2

POST /webhooks/stripe
Valida assinatura HMAC SHA-256.
- 3

UPSERT subscriptions
status=active · current_period_end.
- 4

UPDATE users.plan
pro/premium; cancel → free.

Colunas subscriptions
id · user_id (FK) · stripe_subscription_id · status · price_id · current_period_end · created_at

7

Admin, dados reais e fechamento

8 contas · governança · roteiro de 3 minutos

E-mail admin@admin.com libera painel (WhatsApp, ai_logs, usuários). Senhas nunca aparecem — só hash.

Usuário	E-mail	Senha	Plano
Admin	admin@admin.com	123456	premium
Davi / Leo / Gustavo	*@unicesumar.edu.br	123456	premium
Demos	marina / carlos / juliana / roberto	123456	free/pro

Resumo estratégico (3 min)

Cadastro: 3 docs LGPD → INSERTs ACID · Login: bcrypt + JWT + UNIQUE email + rate limit · Isolamento: WHERE user_id · Crescimento: reset tokens + 2FA tables · Planos: webhook Stripe · Governança: admin + auditoria em consents/logs.

- ✓ Demonstrar no DBeaver: SELECT password_hash (não legível) + user_consent do admin.
- ✓ Login em produção: frontend Vercel → API Railway com CORS *.vercel.app.
- ✓ Documento irmão: TCC_CONTROLAAI_BD_APRESENTACAO_FINAL.pdf (8 págs · 3 capítulos).